Pablo Núñez Fernández — Desarrollo web en entorno cliente (DWEC)

2.1. Selección de un lenguaje de programación de clientes Web

Ejercicio 1. Análisis de Tecnologías Front-end

Tecnología	Categoría
Adobe Flash (ActionScript)	Alternativa histórica obsoleta
TypeScript	Dialecto/superconjunto con transpilación
Java Applets	Alternativa histórica obsoleta
JavaScript (ECMAScript)	Estándar nativo universal
JSX	Dialecto/superconjunto con transpilación
VBScript	Alternativa histórica obsoleta

Ejercicio 2. Justificación de Arquitectura

Dos ventajas técnicas de usar JavaScript nativo en el cliente para reducir el uso de recursos en el servidor:

1. Al ejecutar en el propio navegador las validaciones y los cálculos de interfaz (ordenar listas, mostrar/ocultar elementos, validar un formulario), el servidor no tiene que procesar esas tareas, reduciendo su consumo de CPU.
2. Al intercambiar solo datos puros (normalmente en JSON) en lugar de documentos HTML completos en cada interacción, se reduce el tráfico de red y el ancho de banda que consume el servidor.

Zero Plugins: es el concepto de que JavaScript se ejecuta de forma nativa en cualquier navegador moderno, sin necesidad de instalar ningún complemento externo. Soluciona el problema histórico de tecnologías como los Java Applets o Adobe Flash, que obligaban al usuario a instalar un plugin pesado, no siempre disponible en todos los dispositivos y con importantes problemas de seguridad y rendimiento.

Ejercicio 3. Matriz de Selección de Dialectos

Proyecto	Tecnología recomendada	Justificación
A: plataforma bancaria, 30 desarrolladores, validación estática en desarrollo	**TypeScript**	Su tipado fuerte detecta errores en tiempo de desarrollo/compilación, algo crítico en un equipo grande y en una aplicación financiera donde un fallo puede ser muy costoso.
B: script ligero de 50 líneas para un widget flotante en un sitio estático	**Vanilla JS**	Es un proyecto pequeño que no necesita tipado estático ni paso de compilación; JavaScript nativo se ejecuta directamente y es suficiente.
C: interfaz interactiva basada en componentes reutilizables con ReactJS	**JSX**	Es la sintaxis propia de React para construir componentes mezclando HTML y JS, pensada exactamente para este tipo de arquitectura.

Ejercicio 4. ECMAScript y Transpilación

ECMAScript es la especificación técnica (norma ECMA-262) que define la sintaxis, los tipos y el comportamiento que debe cumplir el lenguaje. JavaScript es la implementación real de esa norma que ejecutan los navegadores.

Un transpilador como Babel traduce código escrito con sintaxis moderna (ES6/ES2015 en adelante) a una versión de JavaScript compatible con motores antiguos que no entienden esa sintaxis nueva. Así, una empresa puede programar con las características modernas del lenguaje sin perder compatibilidad con navegadores desactualizados que todavía usan algunos clientes.

Ejercicio 5. Validación Sintáctica de Identificadores

Identificador	¿Válido?	Motivo si es inválido
`1erUsuario`	❌ Inválido	Empieza por un número
`_totalFactura`	✅ Válido	Empieza por guion bajo
`$elementoDOM`	✅ Válido	Empieza por el símbolo $
`precio-final`	❌ Inválido	El guion - no está permitido (se interpretaría como una resta)
`class`	❌ Inválido	Es una palabra reservada del lenguaje
`montoTotal2`	✅ Válido	Los números solo están prohibidos al principio del identificador
`nombre usuario`	❌ Inválido	Contiene un espacio, no permitido en un identificador
`function`	❌ Inválido	Es una palabra reservada del lenguaje

Ejercicio 6. Detección de Errores por Sensibilidad a Mayúsculas

Código original con errores:

```
Function calcularDescuento(PrecioBase) {
  Const porcentaje = 0.15;
  If (PrecioBase > 100) {
    Alert("Descuento aplicado");
    Return PrecioBase * porcentaje;
  }
  return 0;
}

Console.log(calcularDescuento(150));
```

Errores encontrados (palabras reservadas y objetos globales escritos con mayúscula inicial, cuando JavaScript es *case-sensitive*):

1. `Function` → debe ser `function` (palabra reservada en minúscula).
2. `Const` → debe ser `const`.
3. `If` → debe ser `if`.
4. `Alert` → debe ser `alert` (método del objeto global `window`).
5. `Return` → debe ser `return`.
6. `Console.log` → debe ser `console.log` (el objeto global se llama `console`, en minúscula).

Código corregido:

```
function calcularDescuento(PrecioBase) {
  const porcentaje = 0.15;
  if (PrecioBase > 100) {
    alert("Descuento aplicado");
    return PrecioBase * porcentaje;
  }
  return 0;
}

console.log(calcularDescuento(150));
```

Ejercicio 7. Aplicación de Convenciones Profesionales

Entidad	Convención	Nombre propuesto
Variable con el nombre del cliente actual	camelCase	`nombreClienteActual`
Constante global inmutable con el nº máximo de reintentos	UPPER_SNAKE_CASE	`MAX_REINTENTOS_CONEXION`
Clase que representa un usuario registrado	PascalCase	`UsuarioRegistrado`
Variable con referencia directa a un botón del DOM	prefijo `$`	`$botonEnviar`
Propiedad privada con el token de sesión	prefijo `_`	`_tokenSesion`

Ejercicio 8. Análisis del Comportamiento del ASI

```
function crearConfiguracion() {
  return
  {
    modo: "oscuro",
    puerto: 8080
  };
}

const config = crearConfiguracion();
console.log(config);
```

a) Se imprime `undefined`.

b) Al encontrar un salto de línea justo después de `return`, el mecanismo ASI (Automatic Semicolon Insertion) del motor de JavaScript inserta automáticamente un punto y coma, interpretando la línea como `return;`. El bloque `{ modo: "oscuro", puerto: 8080 }` que viene justo después queda huérfano y ya no se devuelve como objeto, así que la función termina devolviendo `undefined`.

c) Código corregido (la llave de apertura debe ir en la misma línea que `return`, sin salto de línea entre medias):

```
function crearConfiguracion() {
  return {
    modo: "oscuro",
    puerto: 8080
  };
}

const config = crearConfiguracion();
console.log(config);
```

Ejercicio 9. Verificación en Consola y Ámbito

```
let totalVentas = 500;
let TotalVentas = 1200;

console.log(totalVentas);
console.log(TotalVentas);
```

Salida: primero `500` y después `1200`.

JavaScript es *case-sensitive*, por lo que `totalVentas` y `TotalVentas` son dos identificadores completamente independientes (dos variables distintas en memoria), aunque solo se diferencien por la mayúscula de la "T". No hay ningún conflicto ni redeclaración entre ellas: cada `console.log` imprime el valor de su propia variable.
